

## 1. Difference between static and dynamic Hashing. \*\*\*

### Difference between Static Hashing and Dynamic Hashing

#### Static Hashing

The number of buckets is **fixed**.

The same key always maps to the **same bucket**.

Bucket overflow is **more common**.

No directory is used.

Simple and easy to implement.

Best for **small or stable databases**.

#### Dynamic Hashing

The number of buckets can **increase or decrease dynamically**.

The bucket may change after **bucket splitting or directory expansion**.

Bucket overflow is **reduced** by splitting buckets.

Uses a **directory** to locate buckets.

More complex but more efficient for large databases.

Best for **large and growing databases**.

## 2. Basic Structure of Extensible Hashing?

### Basic Structure of Extensible Hashing

The basic structure of **Extensible Hashing** consists of the following components:

#### 1. Directories

- Directories store **pointers to buckets**.
- Each directory has a unique **binary ID**.
- The hash function returns the directory ID to locate the correct bucket.
- Whenever directory expansion occurs, the directory IDs may change.
- **Number of Directories =  $2^{\text{Global Depth}}$** .

#### Definition:

A **Directory** is a collection of pointers that stores the addresses of buckets. Each directory has a unique binary ID used by the hash function to locate the correct bucket.

#### 2. Buckets

- Buckets store the **hashed keys (records)**.
- Directories point to the buckets.

- A bucket may be pointed to by more than one directory if its **Local Depth** is less than the **Global Depth**.

**Definition:**

A **Bucket** is a storage unit that holds one or more hashed keys (records). It is accessed through directory pointers.

### 3. Global Depth

- Global Depth is associated with the **directories**.
- It represents the **number of bits** used by the hash function to identify a directory.
- **Global Depth = Number of bits in the Directory ID.**

**Definition:**

**Global Depth** is the number of bits used by the hash function to identify a directory. It is associated with the directory.

### 4. Local Depth

- Local Depth is associated with the **buckets**.
- It represents the number of bits used for a bucket.
- It is used to decide what action should be taken when a **bucket overflow** occurs.
- **Local Depth is always less than or equal to Global Depth.**

**Definition:**

**Local Depth** is the number of bits associated with a bucket. It is used to determine the action taken when a bucket overflow occurs.

### 5. Bucket Splitting

- When a bucket becomes **full**, it is divided into **two buckets**.
- This process is called **Bucket Splitting**.

**Definition:**

**Bucket Splitting** is the process of dividing a full bucket into two separate buckets when bucket overflow occurs

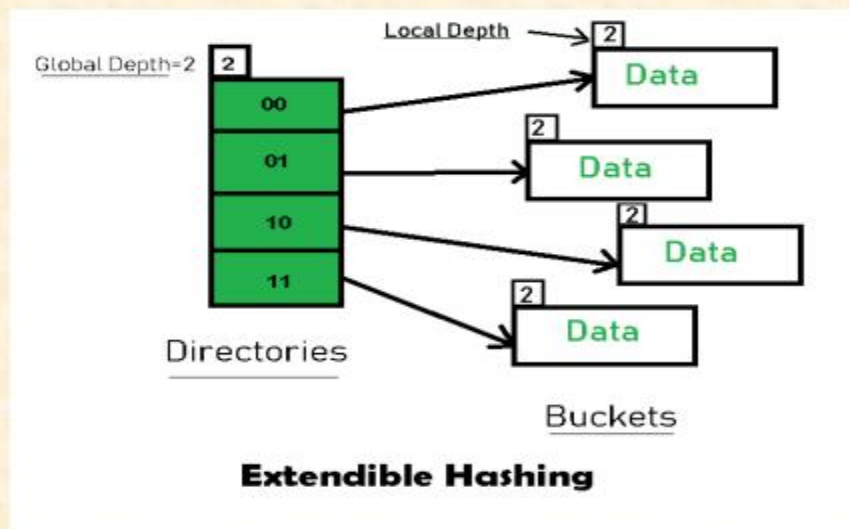
### 6. Directory Expansion

- Directory Expansion occurs when a bucket overflows **and Local Depth is equal to Global Depth**.
- In this case, the directory size is **doubled**, and then the bucket is split.

**Definition:**

**Directory Expansion** is the process of increasing (doubling) the number of directories when a bucket overflows and the **Local Depth is equal to the Global Depth**.

### ❑ Basic Structure of Extendible Hashing



**3. Definition:**

- What is directory?
- What is bucket?
- What is Local Path?
- What is Global Path?

See 2 no question answer.

**4. Basic Working of Extendible Hashing?**

Basic Working of Extendible Hashing –

**Step 1 – Analyze Data Elements:** Data elements may exist in various forms eg. Integer, String, Float, etc. Currently, let us consider data elements of type integer. eg: 49.

**Step 2 – Convert into binary format:** Convert the data element in Binary form. For string elements, consider the ASCII equivalent integer of the starting character and then convert the integer into binary form. Since we have 49 as our data element, its binary form is 110001.

**Step 3 – Check Global Depth of the directory:** Suppose the global depth of the Hash-directory is 3.

**Step 4 – Identify the Directory:** Consider the ‘Global-Depth’ number of LSBs in the binary number and match it to the directory id. For Example, the binary value of 49 is 110001 and the global-depth is 3. So, the hash function will return 3 LSBs of 110001 that is, 001.

**Step 5 – Navigation:** Now, navigate to the bucket pointed by the directory with directory-id 001

**Step 6 – Insertion and Overflow Check:** Insert the element and check if the bucket overflows. If an overflow is encountered, go to step 7 followed by Step 8, otherwise, go to step 9.

**Step 7 – Tackling Overflow Condition during Data Insertion:** Many times, while inserting data in the buckets, it might happen that the Bucket overflows. In such cases, we need to follow an appropriate procedure to avoid mishandling of data.

First, Check if the local depth is less than or equal to the global depth. Then choose one of the cases below.

**Case-1:** If the local depth of the overflowing Bucket **is equal to** the global depth, then Directory Expansion, as well as Bucket Split, needs to be performed. Then increment the global depth and the local depth value by 1. And, assign appropriate pointers. Directory expansion will double the number of directories present in the hash structure.

**Case-2:** In case the local depth **is less than** the global depth, then only Bucket Split takes place. Then increment only the local depth value by 1. And, assign appropriate pointers.

**Step 8 – Rehashing of Split Bucket Elements:** The Elements present in the overflowing bucket that is split are rehashed w.r.t the new global depth of the directory.

**Step 9** – The element is successfully hashed

